3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

Classes : Offline & Online

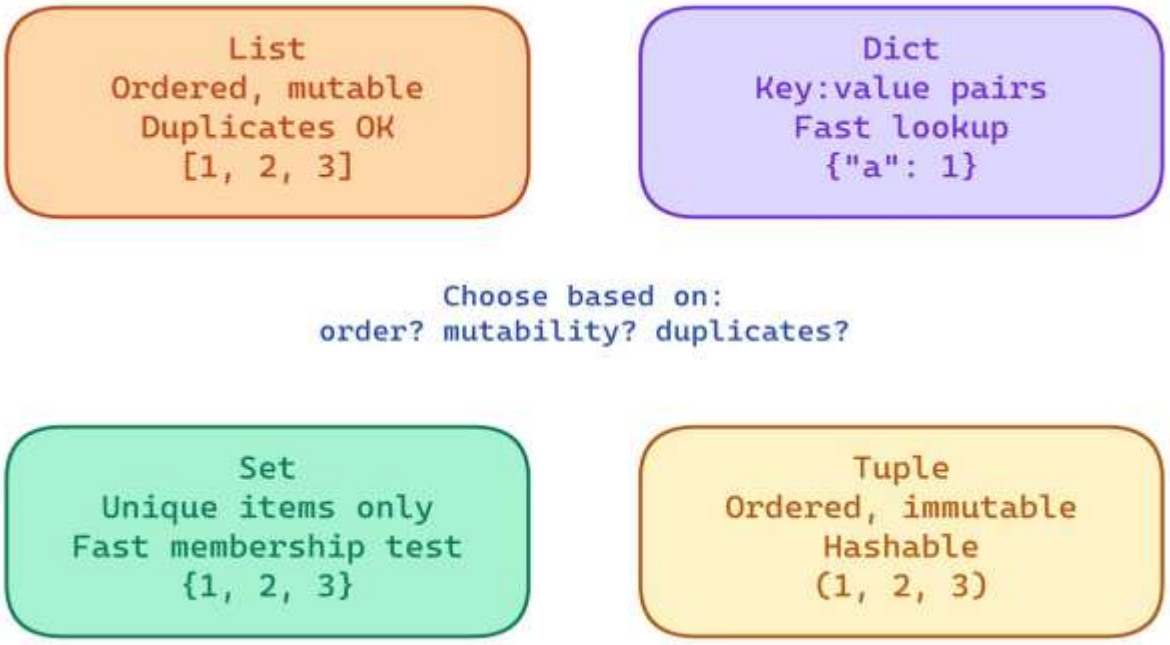
Daily Teaching : 2 Hours

Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 6 – Tuples & Sets

T FOUNDATION



Learning Objectives

By the end of this class, students will be able to:

- Understand what **tuples** are.
- Understand what **sets** are.
- Identify the differences between lists, tuples, and sets.
- Create and access tuples.
- Understand tuple immutability.
- Use tuple methods.
- Create and modify sets correctly.
- Add and remove elements from sets.
- Perform union, intersection, difference, and symmetric difference.
- Identify duplicate values.
- Remove duplicate data using sets.
- Apply duplicate removal to real-world data.

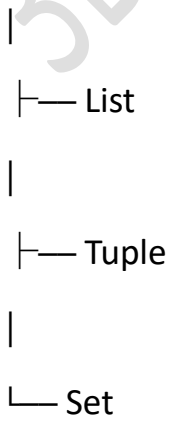
3DWEBSOFT FOUNDATION

1. Introduction – Tuple vs Set

Python provides different collection types for storing multiple values.

Today we focus on:

Python Collections



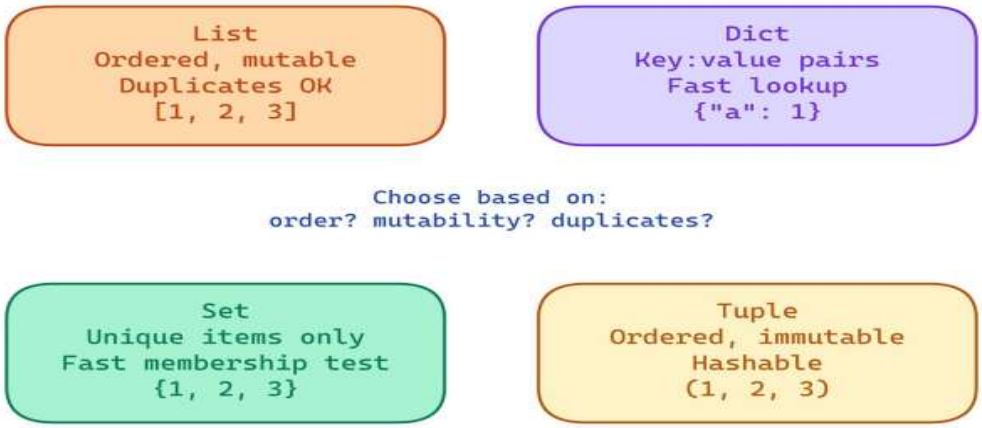
A simple way to remember them:

Collection	Ordered	Mutable	Duplicates
List	✅ Yes	✅ Yes	✅ Allowed
Tuple	✅ Yes	❌ No	✅ Allowed
Set	❌ No*	✅ Yes	❌ Not allowed

*A set does not provide positional indexing/order for normal access.

LIST VS TUPLE VS SET VS DICTIONARY in Python

FEATURE	LIST	TUPLE	SET	DICTIONARY
Syntax	[1, 2, 3]	(1, 2, 3)	{1, 2, 3}	{"a": 1, "b": 2}
Ordered?	✅ Yes	✅ Yes	❌ No	✅ Yes
Mutable?	✅ Yes	❌ No	✅ Yes	✅ Yes
Duplicates?	✅ Allowed	✅ Allowed	❌ Not Allowed	✅ Keys unique
Use Case	General collection	Fixed data	Unique items	Key-value mapping



PART A – TUPLES

2. What is a Tuple?

A **tuple** is an ordered collection of elements that cannot normally be changed after it is created.

Tuples are written using **parentheses ()**.

Example:

subjects = ("Python", "Maths", "Physics", "English")

We can access elements using indexes:

print(subjects[0])

print(subjects[2])

Output:

Python

Physics

3. Creating Tuples

Empty Tuple

data = ()

Tuple of Numbers

3DWEBSOFT FOUNDATION

marks = (85, 90, 78, 92, 88)

Tuple of Strings

colors = ("Red", "Blue", "Green")

Mixed Tuple

student = ("Aisha", 17, 92.5)

4. Tuple Indexing

Like lists, tuple indexing starts from 0.

marks = (85, 90, 78, 92, 88)

Index	Value
0	85
1	90
2	78
3	92
4	88

Example:

print(marks[0])

print(marks[3])

Output:

85

92

5. Negative Indexing in Tuples

Tuples support negative indexing.

3DWEBSOFT FOUNDATION

```
marks = (85, 90, 78, 92, 88)


print(marks[-1])
```

Output:

88

```
Positive:  0   1   2   3   4
           ↓   ↓   ↓   ↓   ↓
Tuple:     85  90  78  92  88
           ↑   ↑   ↑   ↑   ↑
Negative: -5  -4  -3  -2  -1
```

6. Tuple Slicing

Tuples support slicing just like lists.

```
marks = (85, 90, 78, 92, 88)


print(marks[1:4])
```

Output:

(90, 78, 92)

Reverse:

```
print(marks[::-1])
```

Output:

(88, 92, 78, 90, 85)

3DWEBSOFT FOUNDATION

Tuple = (0, 1, 2, 3, 4, 5)

0	1	2	3	4	5

Tuple[0] = 0

Tuple[1] = 1

Tuple[2] = 2

Tuple[3] = 3

Tuple[4] = 4

Tuple[5] = 5

Tuple[0:] = (0, 1, 2, 3, 4, 5)

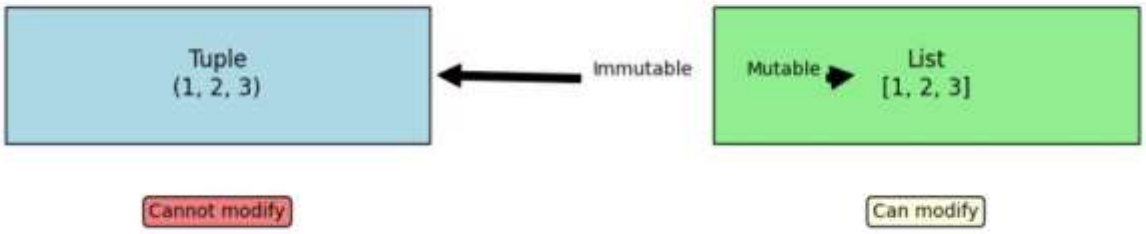
Tuple[:] = (0, 1, 2, 3, 4, 5)

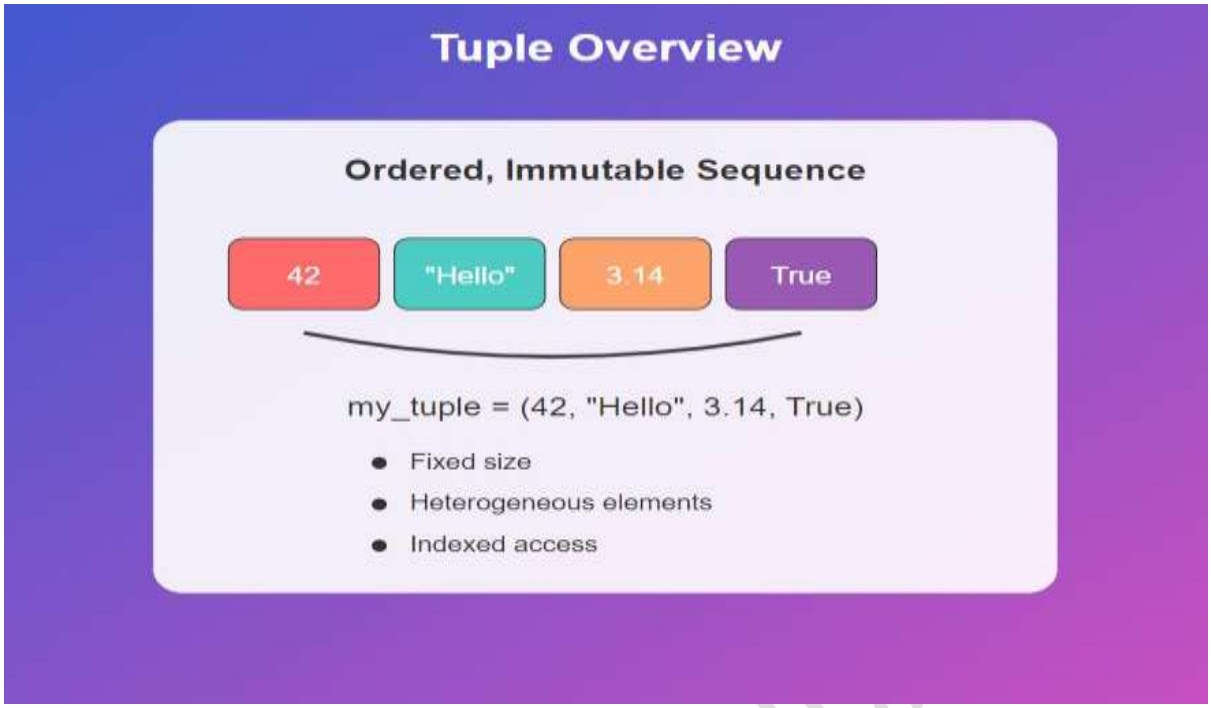
Tuple[2:4] = (2, 3)

Tuple[1:3] = (1, 2)

Tuple[:4] = (0, 1, 2, 3)

Immutability of Tuples vs. Mutability of Lists





7. Why Are Tuples Immutable?

Immutable means the tuple cannot be changed after creation.

Example:

marks = (80, 90, 70)

marks[1] = 95

This produces an error because tuples cannot be modified this way.

TypeError

Why is this useful?

Tuples are useful when data should remain fixed.

Examples:

- Coordinates
- Days of the week
- Months

3DWEBSOFT FOUND

- Fixed configuration values
- Student ID information
- RGB color values

Example:

coordinates = (17.3850, 78.4867)

The values can be treated as a fixed pair.

8. Tuple Methods

Tuples have fewer methods than lists because they cannot be changed.

count()

Counts how many times a value occurs.

marks = (80, 90, 80, 70, 80)

```
print(marks.count(80))
```

Output:

3

index()

Finds the position of a value.

marks = (80, 90, 70)

```
print(marks.index(90))
```

Output:

1

9. Tuple Packing

3DWEBSOFT FOUNDATION

Python allows us to create a tuple without explicitly writing parentheses.

student = "Aisha", 17, 92

Python treats it as:

student = ("Aisha", 17, 92)

This is called **tuple packing**.

10. Tuple Unpacking

We can assign tuple elements to separate variables.

student = ("Aisha", 17, 92)

name, age, marks = student

print(name)

print(age)

print(marks)

Output:

Aisha

17

92

This is called **tuple unpacking**.

3DWEBSOFT FOUNDATION

PART B – SETS

11. What is a Set?

A **set** is a collection used to store **unique values**.

The most important feature:

A set automatically removes duplicate values.

Sets are written using curly braces {}.

Example:

numbers = {10, 20, 30, 40}

If we create:

numbers = {10, 20, 10, 30, 20, 40}

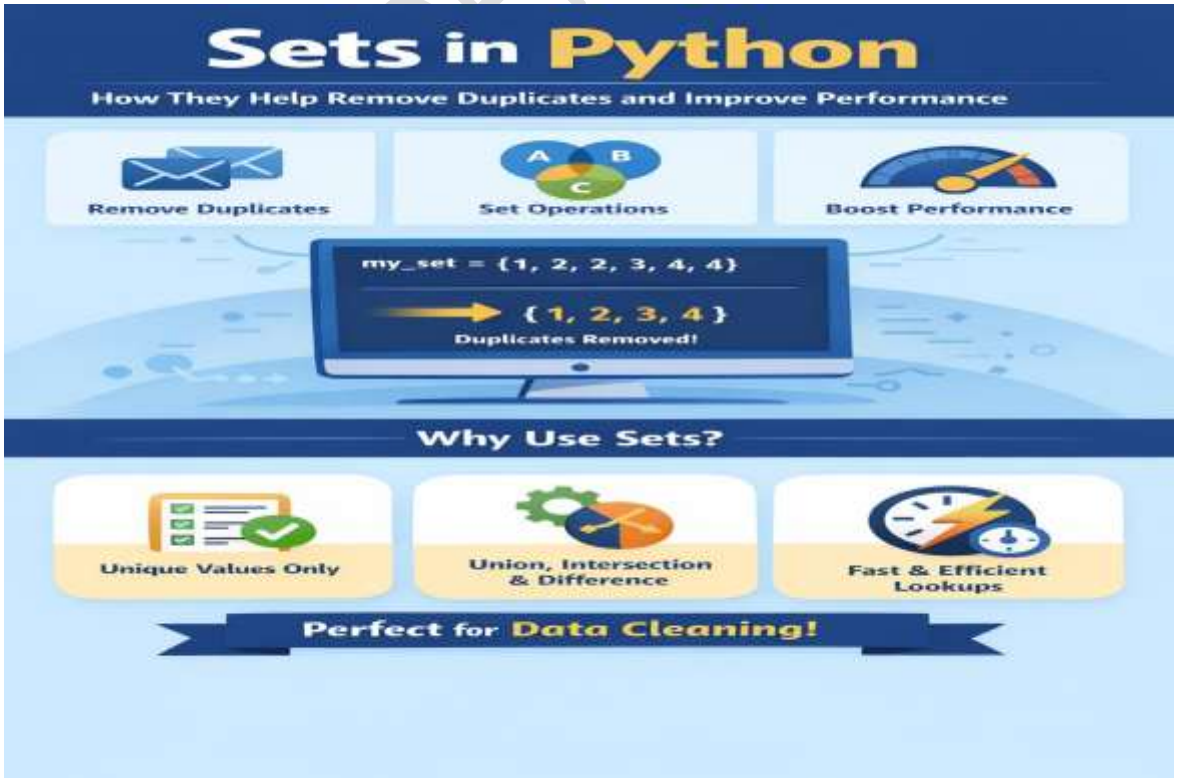
Python stores only unique values.

Original:

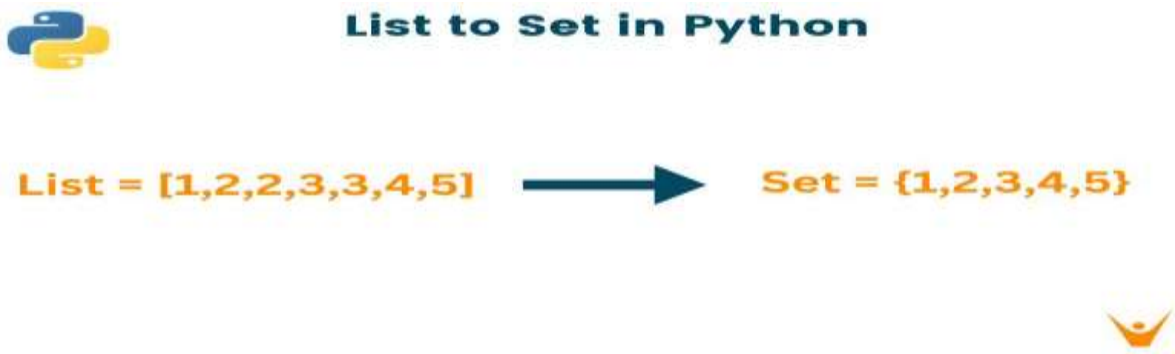
10 20 10 30 20 40

↓ SET

10 20 30 40



SET FOUNDATION



12. Creating a Set

numbers = {10, 20, 30, 40}

print(numbers)

A set does not use indexes like a list.

Therefore, this is not valid:

numbers[0]

Sets are designed for **membership and set operations**, rather than positional access.

13. Important: Creating an Empty Set

This is a common beginner mistake.

data = {}

This creates an **empty dictionary**, not an empty set.

To create an empty set:

data = set()

Remember:

{} → Empty dictionary

set() → Empty set

3DWEBSOFT FOUNDATION

14. Sets Automatically Remove Duplicates

Example:

numbers = {10, 20, 10, 30, 20, 40}

print(numbers)

The result contains only:

10

20

30

40

The exact display order should not be relied upon.

15. Creating a Set from a List

This is extremely useful for duplicate removal.

marks = [80, 90, 80, 70, 90, 85]

unique_marks = set(marks)

print(unique_marks)

The duplicate values are removed.

Conceptually:

List

↓

[80, 90, 80, 70, 90, 85]

3DWEBSOFT FOUNDATION

↓

set()

↓

{80, 90, 70, 85}

16. Adding Elements to a Set

Use add().

students = {"Aisha", "Rahul", "Priya"}

students.add("Arjun")

print(students)

17. Adding Multiple Elements

Use update().

students = {"Aisha", "Rahul"}

students.update(["Priya", "Arjun", "Sara"])

print(students)

Now the set contains all unique student names.

18. Removing Elements from a Set

remove()

3DWEBSOFT FOUNDATION

students = {"Aisha", "Rahul", "Priya"}

students.remove("Rahul")

If the value does not exist, remove() raises an error.

discard()

students = {"Aisha", "Rahul", "Priya"}

students.discard("Arjun")

If "Arjun" does not exist, discard() does not raise an error.

Remember

remove() → error if item doesn't exist

discard() → safely ignores missing item

pop()

pop() removes an arbitrary element from a set.

students = {"Aisha", "Rahul", "Priya"}

students.pop()

Because sets are not positional collections, do **not** assume which element will be removed.

clear()

students.clear()

This removes all elements.

3DWEBSOFT FOUNDATION

19. Checking Membership

The in operator works very well with sets.

students = {"Aisha", "Rahul", "Priya"}

print("Aisha" in students)

Output:

True

Example:

print("Arjun" in students)

Output:

False

PART C – SET OPERATIONS

Set operations are one of the most important concepts in this lesson.

Suppose:

python_students = {"Aisha", "Rahul", "Priya", "Arjun"}

ai_students = {"Priya", "Arjun", "Sara", "John"}

20. Union

Union combines all unique elements from both sets.

Symbol:

U

Python:

all_students = python_students.union(ai_students)

3DWEBSOFT FOUNDATION

print(all_students)

Or:

all_students = python_students | ai_students

Concept:

Python Students

∪

AI Students

↓

All Unique Students

21. Intersection

Intersection finds elements common to both sets.

Symbol:

∩

Python:

common = python_students.intersection(ai_students)

print(common)

Or:

common = python_students & ai_students

Result:

Priya

Arjun

3DWEBSOFT FOUNDATION

22. Difference

Difference finds elements that exist in one set but not in the other.

python_students = {"Aisha", "Rahul", "Priya", "Arjun"}

ai_students = {"Priya", "Arjun", "Sara"}

result = python_students.difference(ai_students)

print(result)

This identifies students who are in the Python group but not the AI group.

Shortcut:

result = python_students - ai_students

23. Symmetric Difference

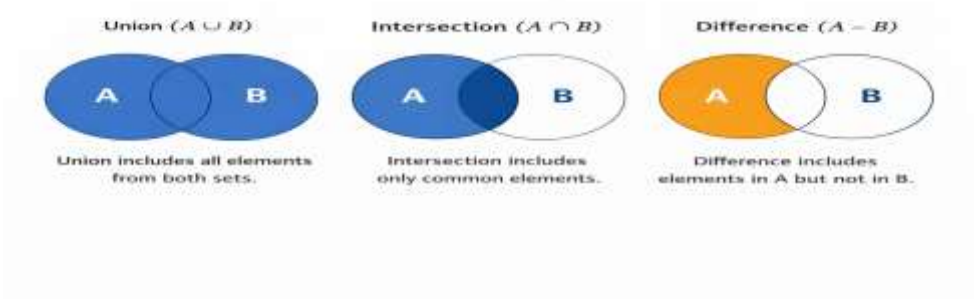
This returns elements that are in either set, but **not in both**.

result = python_students.symmetric_difference(ai_students)

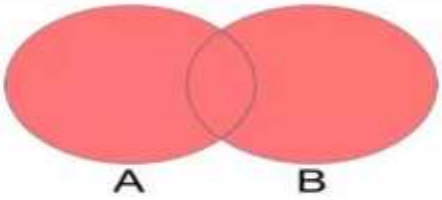
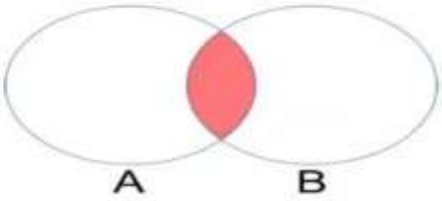
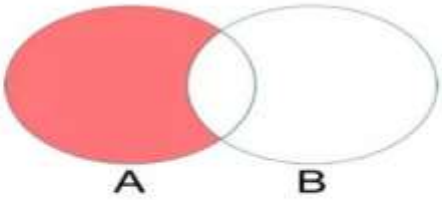
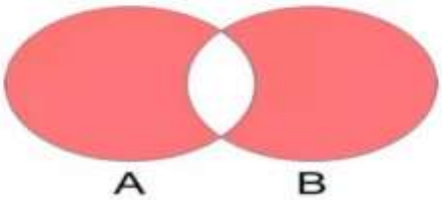
print(result)

Shortcut:

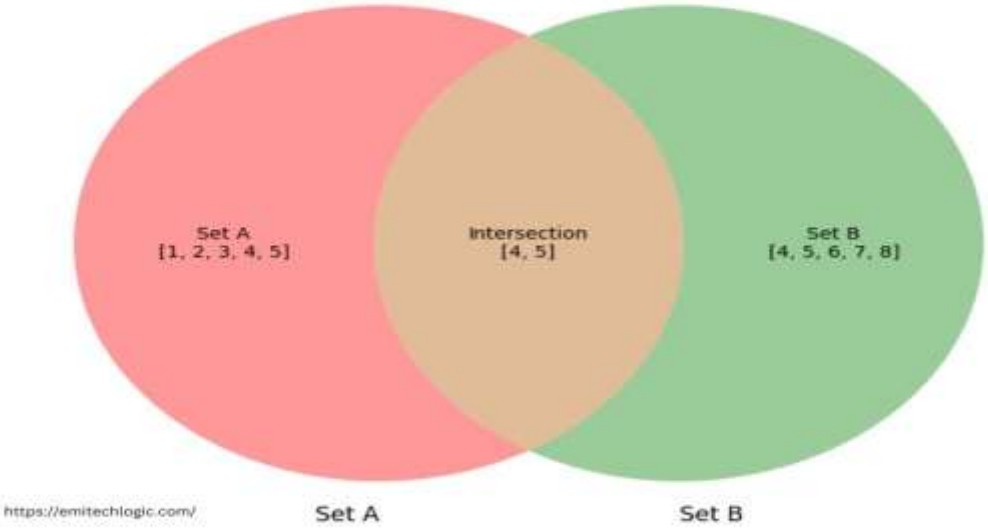
result = python_students ^ ai_students



3DWEBSOFT FOUNDATION

Set Operation	Venn Diagram	Interpretation
Union		$A \cup B$, is the set of all values that are a member of A, or B, or both.
Intersection		$A \cap B$, is the set of all values that are members of both A and B.
Difference		$A \setminus B$, is the set of all values of A that are not members of B
Symmetric Difference		$A \triangle B$, is the set of all values which are in one of the sets, but not both.

Concept of Python Sets: Union, Intersection, and Difference



24. Set Operations Summary

Operation	Meaning	Python
Union	All unique elements	A | B
Intersection	Common elements	A & B
Difference	Elements only in A	A - B
Symmetric Difference	Elements in either, not both	A ^ B

Practical Session

PRACTICAL 1 – Remove Duplicate Marks

Problem

Given:

marks = [85, 90, 85, 78, 90, 88, 78]

Remove duplicates.

Solution

```
marks = [85, 90, 85, 78, 90, 88, 78]


unique_marks = set(marks)


print("Original:", marks)

print("Unique:", unique_marks)
```

Expected concept:

Original:

85 90 85 78 90 88 78

3DWEBSOFT FOUNDATION

Unique:

85 90 78 88

PRACTICAL 2 – Remove Duplicate Student Names

```
students = [
    "Aisha",
    "Rahul",
    "Priya",
    "Aisha",
    "Arjun",
    "Rahul"
]

unique_students = set(students)

print("Original Students:", students)
print("Unique Students:", unique_students)
```

This can be useful when cleaning attendance or registration data.

PRACTICAL 3 – Find Duplicate Values

A useful technique is to compare the list length with the set length.

```
marks = [80, 90, 80, 70, 90, 85]

if len(marks) != len(set(marks)):
```

3DWEBSOFT FOUNDATION

```
    print("Duplicates found")
else:
    print("No duplicates")
```

Output:

Duplicates found

Why does this work?

Suppose:

List length = 6

Set length = 4

Since the set removed duplicates, the lengths are different.

PRACTICAL 4 – Find Duplicate Student Names

```
students = [
    "Aisha",
    "Rahul",
    "Priya",
    "Aisha",
    "Rahul",
    "Sara"
]

unique_students = set(students)

duplicates = []
```

3DWEBSOFT FOUNDATION

```
for student in unique_students:
    if students.count(student) > 1:
        duplicates.append(student)

print("Duplicate Students:", duplicates)
```

Possible output:

Duplicate Students: ['Aisha', 'Rahul']

PRACTICAL 5 – Unique Subject Names

```
subjects = [
    "Python",
    "Maths",
    "Python",
    "Physics",
    "Maths",
    "AI"
]

unique_subjects = set(subjects)

print("Unique Subjects:")
for subject in unique_subjects:
    print(subject)
```

PRACTICAL 6 – Student Club Analysis

3DWEBSOFT FOUNDATION

Suppose two groups exist:

coding = {"Aisha", "Rahul", "Priya", "Arjun"}

robotics = {"Priya", "Arjun", "Sara", "John"}

Students in both clubs

both = coding & robotics

print("Students in both:", both)

Students in at least one club

all_students = coding | robotics

print("Students in either club:", all_students)

Only coding students

only_coding = coding - robotics

print("Only Coding:", only_coding)

PRACTICAL 7 – Duplicate Phone/ID Data

Consider student IDs:

student_ids = [101, 102, 103, 101, 104, 102, 105]

Remove duplicates:

unique_ids = set(student_ids)

print("Unique IDs:", unique_ids)

This is an example of basic **data cleaning**.

3DWEBSOFT FOUNDATION

Mini Project – Student Data Cleaner

Problem Statement

A school has collected student names from different registration forms. Some names appear multiple times.

Create a program that:

1. Stores all names in a list.
2. Finds the number of registrations.
3. Removes duplicates.
4. Finds the number of unique students.
5. Displays the unique student names.

Program

```
students = [
    "Aisha",
    "Rahul",
    "Priya",
    "Aisha",
    "Arjun",
    "Rahul",
    "Sara",
    "Priya"
]


unique_students = set(students)


print("===== STUDENT DATA REPORT =====")
```

3DWEBSOFT FOUNDATION

```
print("Total registrations:", len(students))
print("Unique students:", len(unique_students))

print("\nUnique Student Names:")

for student in unique_students:
    print(student)
```

Concept

Raw Data

↓

List

↓

Duplicate Values

↓

set()

↓

Unique Data

↓

Clean Dataset

3DWEBSOFT FOUNDATION

Advanced Challenge – Keep Original Order

A normal set() removes duplicates, but a set should not be used when you need to preserve the original order.

For example:

```
students = [
    "Aisha",
    "Rahul",
    "Aisha",
    "Priya",
    "Rahul"
]

unique_students = list(dict.fromkeys(students))

print(unique_students)
```

Output:

['Aisha', 'Rahul', 'Priya']

This technique removes duplicates while preserving the first occurrence order.

Practical Comparison

Ask students to run:

```
marks = [90, 80, 90, 70, 80]

print("List:", marks)
print("Tuple:", tuple(marks))
print("Set:", set(marks))
```

They should observe:

List

→ ordered

3DWEBSOFT FOUNDATION

→ duplicates allowed

→ mutable

Tuple

→ ordered

→ duplicates allowed

→ immutable

Set

→ unique values

→ no positional indexing

→ mutable

Interactive Classroom Activity – "Data Cleaner"

Give students:

```
data = [
    "Python",
    "AI",
    "Python",
    "ML",
    "Cybersecurity",
    "AI",
    "Python",
    "ML"
]
```

3DWEBSOFT FOUNDATION

Ask them:

Challenge 1

How many total values are there?

Challenge 2

How many unique values are there?

Challenge 3

Remove duplicates.

Challenge 4

Display the unique technologies.

Challenge 5

Add "Data Science" to the set.

Challenge 6

Check whether "Python" exists.

3DWEBSOFT FOUNDATION

Quiz

1. What is a tuple?

A. A mutable collection
B. An ordered, immutable collection
C. A collection that only stores numbers
D. A loop

Answer: B

2. Which brackets are normally used for tuples?

Answer: ()

3. Can we change a tuple after creation?

Answer: No.

4. What is the main feature of a set?

Answer: It stores unique values and automatically eliminates duplicates.

5. Which brackets are used to create a set?

Answer: {}

6. How do you create an empty set?

Answer:

set()

7. Which operation finds common elements?

Answer: Intersection.

8. Which symbol represents union?

Answer: |

9. Which symbol represents intersection?

Answer: &

10. Which function converts a list into a set?

Answer:

set()

Homework

Task 1 – Duplicate Marks

Given:

marks = [90, 80, 90, 70, 80, 95, 70]

Find:

- Total number of marks
- Unique marks
- Number of unique marks
- Whether duplicates exist

3DWEBSOFT FOUNDATION

Task 2 – Student Names

Given:

```
students = [
    "Aisha",
    "Rahul",
    "Priya",
    "Aisha",
    "Arjun",
    "Rahul"
]
```

Write a program to:

- Remove duplicates
- Count unique students
- Display unique names

Task 3 – Club Members

Create:

```
python_club = {"Aisha", "Rahul", "Priya"}
ai_club = {"Priya", "Arjun", "Sara"}
```

Find:

- Students in both clubs
- Students in either club
- Students only in Python club
- Students only in AI club

3DWEBSOFT FOUNDATION

2-Hour Class Plan

Time	Topic
0–10 min	Revision of Lists
10–25 min	Introduction to Tuples
25–40 min	Tuple indexing, slicing & immutability
40–55 min	Tuple methods & unpacking
55–70 min	Introduction to Sets
70–85 min	Duplicate-data removal
85–100 min	Set operations
100–110 min	Student Data Cleaner project
110–116 min	Interactive challenge
116–120 min	Quiz & homework

Real-World Applications

Tuples

Tuples are useful for fixed information such as:

Coordinates

(17.3850, 78.4867)

RGB Color

(255, 100, 50)

Date

3DWEBSOFT FOUNDATION

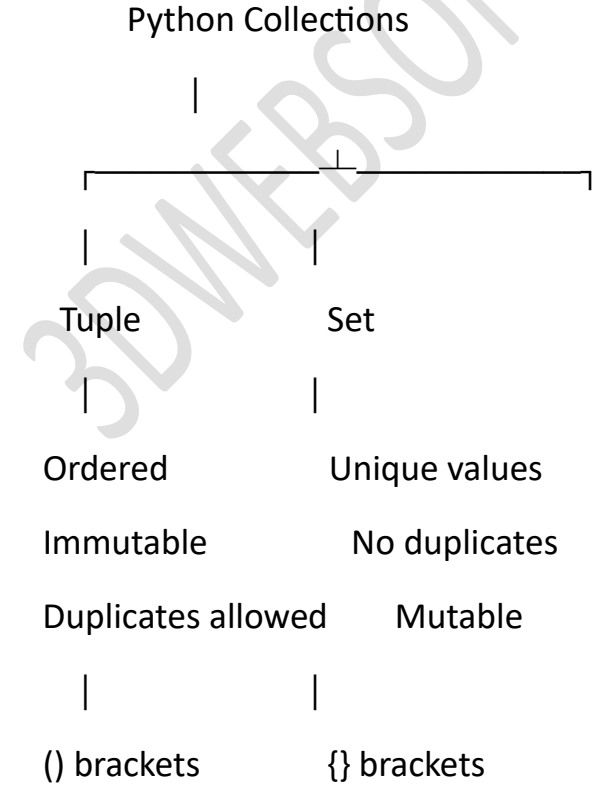
(2, 9, 2026)

Sets

Sets are especially useful for:

- Removing duplicate records
- Checking membership
- Comparing datasets
- Finding common students
- Data cleaning
- Finding unique categories
- Comparing course registrations
- Basic data analysis
- Preparing datasets for AI/ML

Quick Revision



3DWEBSOFT FOUNDATION

Most Important Code

```
# Tuple
marks = (80, 90, 85)

# Set
unique_marks = {80, 90, 85}

# List → Set
marks = [80, 90, 80, 85, 90]
unique_marks = set(marks)

# Add
unique_marks.add(95)

# Membership
print(90 in unique_marks)

# Union
A | B

# Intersection
A & B

# Difference
A - B
```

3DWEBSOFT FOUNDATION

Symmetric Difference

A ^ B

Key Takeaway

Tuple = fixed collection of data.
Set = unique collection of data.

For today's practical:

Raw student data → List → set() → Duplicate removal → Clean/unique data

3DWEBSOFT FOUNDATION